Deep Learning for Anomaly Detection in Cybersecurity

AIML339

22/10/2025

By Sahil Adwani

Motivation

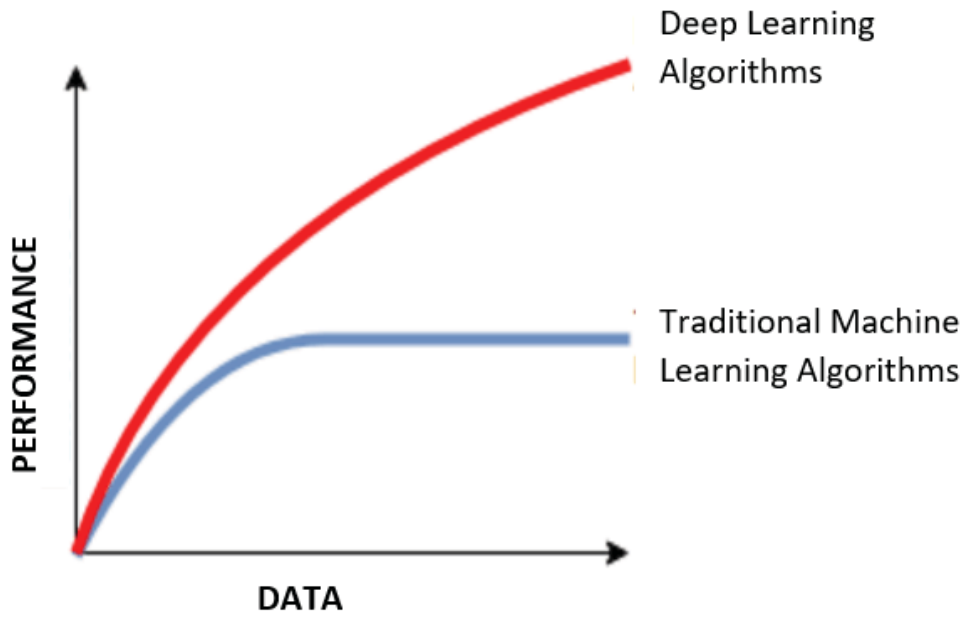
- **Why anomaly detection is important in cybersecurity?**
  - In 2024, cyber threats grew sharply with 31% more reported vulnerabilities and 20% more active attacks. This means it’s getting easier for hackers to find ways in.
  - Detecting unusual patterns like a sudden spike in traffic or unauthorized access is critical to stopping threats.
- **Why traditional systems fall short?**
  - Traditional systems often rely on known signatures or manual rules, which fail to catch new or subtle attacks.
- **The solution: Anomaly detection**
  - learns patterns of normal behavior and flags anything that doesn’t fit, making it well-suited for detecting new or evolving cyber threats.

Motivation

- **Why use only normal data to train?**
  - In real-world cybersecurity, you have plenty of normal data. But labeled attack data is rare, unlabeled, or unknown.
  - These models are designed to learn the normal behavior of a system and then detect anything that deviates from it as an anomaly.
  - This approach is practical and scalable because it doesn't require examples of every possible attack. Hence model can detect unseen attacks.

Motivation

- **Why deep learning methods instead of traditional machine learning methods?**
  - Traditional ML methods require manual feature engineering, struggle with high dimensional or complex data and may miss subtle attacks
  - Deep Learning methods learn patterns automatically from raw data, handles large, noisy, complex datasets and better at detecting evolving or unknown threats.



Research Background and Contribution

- Although deep learning-based anomaly detection is a promising approach, its use in cybersecurity applications is still underexplored.
- Most existing studies have tested deep learning models on image-based datasets, not on structured cybersecurity data.
- This project helps fill that gap by testing and comparing deep learning models on a real-world cybersecurity dataset, and provides practical insights and reproducible steps others can follow.

The Goal

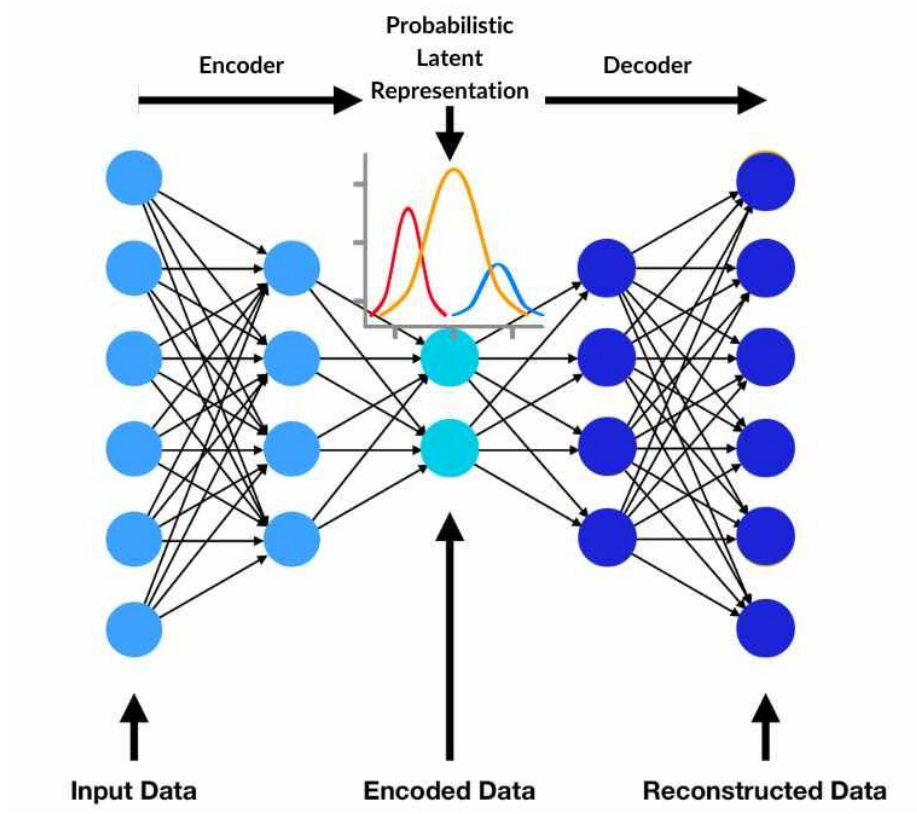
- **Goal**: A fair and systematic comparison of three deep learning approaches on realistic cybersecurity data to identify the most effective one for detecting anomalies in real-world traffic.

- **Question:** Which model is most effective for real-world anomaly detection in cybersecurity?

Selected Models

- This study compares three distinct deep learning approaches:
  - **Variational Autoencoder (VAE)**
  - **GANomaly**
  - **Deep Support Vector Data Description (Deep SVDD)**

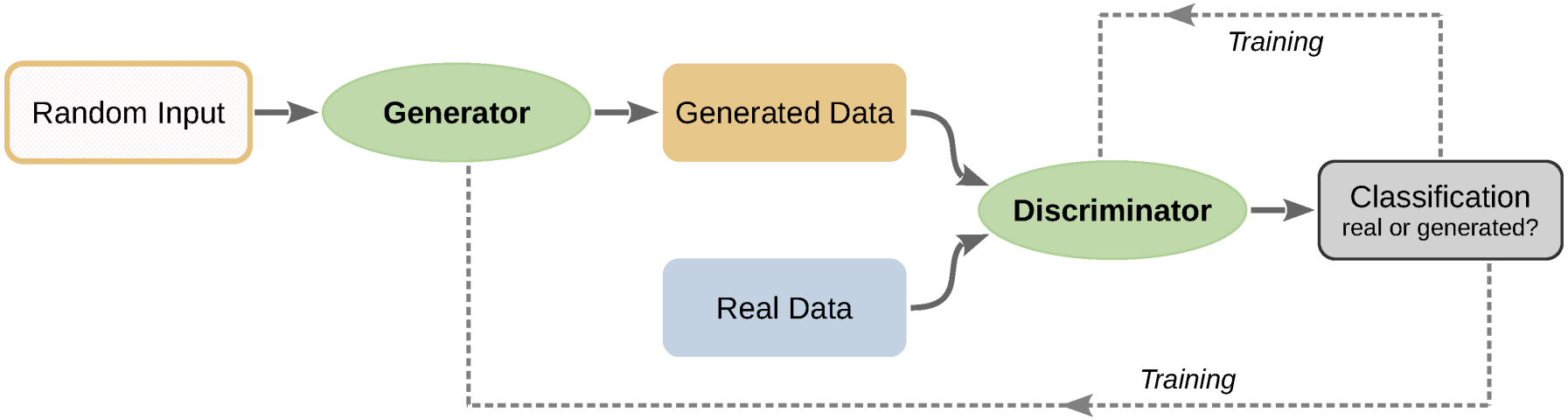
VAE


Probabilistic
Latent
Representation
Encoder
Decoder
Input Data
Encoded Data
Reconstructed Data

VAE

- **Core Idea:** Learns a flexible "blueprint" of normal data and flags anything it can't properly reconstruct.
- **How It Works:**
- **Encoder:**
  - Compresses a normal network flow into a **probability distribution** in a latent space.
  - *Unlike a standard autoencoder that uses a fixed point, this creates a flexible understanding of normal data.*
- **Latent Space:**
  - The "learned blueprint" of normal traffic, capturing essential patterns.
- **Decoder:**
  - Samples from this distribution and attempts to reconstruct the original input.
- **How It Detects Anomalies:**
- Becomes an expert at reconstructing normal traffic.
- When an attack flow passes through:
  - The encoder **can't map it** well to the normal blueprint.
  - The decoder **struggles to reconstruct** it accurately.
- **The Anomaly Score is the RECONSTRUCTION ERROR.**
  - A high error means "I've never seen this before" → Flag as an attack.

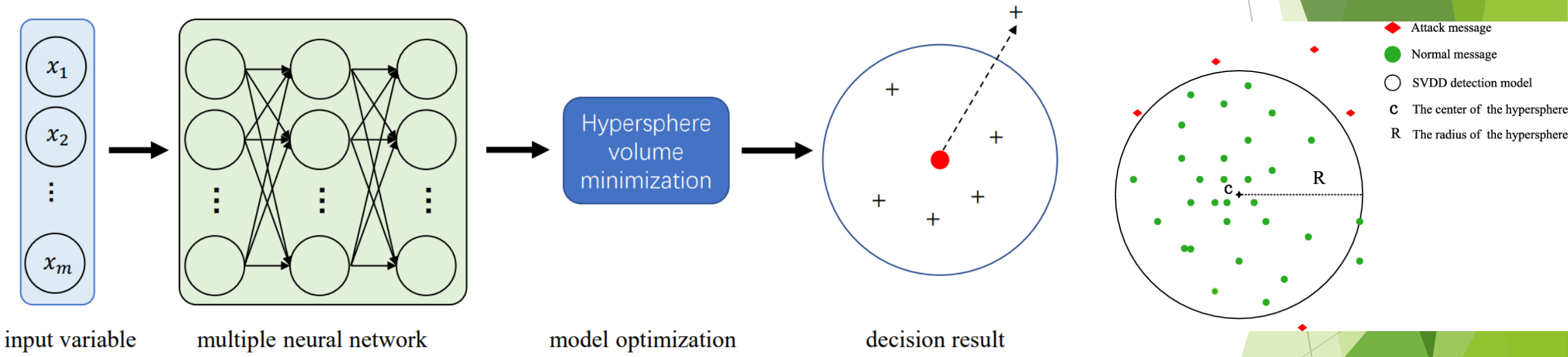
GANomaly


Random Input
Generator
Generated Data
Real Data
Discriminator
Classification
real or generated?
Training
Training

GANomaly

- **Core Idea:** Uses an adversarial game to become hyper-sensitive to subtle anomalies by comparing the "idea" of the original with its reconstruction.
- **How It Works: Two Networks Compete**
- **Generator (The Forger):** Encoder -> Decoder -> Encoder
  - Takes an input, encodes it (latent_z), decodes it (reconstruction), then re-encodes the reconstruction (latent_ž).
  - **Goal:** Make perfect reconstructions that fool the discriminator.
- **Discriminator (The Critic):**
  - Looks at any latent code and decides: **"Is this from a REAL input or a GENERATED reconstruction?"**
  - This adversarial training forces the generator to master normal data.
- **How It Detects Anomalies:**
- The anomaly score is a combination of:
- **Reconstruction Error:** How pixel-perfect is the reconstruction?
- **Latent Space Discrepancy (The Key):** How different is the original "idea" (latent_z) from the reconstruction's "idea" (latent_ž)?
  - **Normal Data:** The two latent codes are very similar.
  - **Anomalous Data:** The generator gets confused; the two latent codes are very different.
- **Flags inputs that are both hard to reconstruct AND whose core "idea" changes.**

Deep SVDD


x_1
x_2
x_m
Hypersphere volume minimization
Attack message
Normal message
SVDD detection model
c The center of the hypersphere
R The radius of the hypersphere
c
R
input variable
multiple neural network
model optimization
decision result

Deep SVDD

- **Core Idea:** Learns a simple, compact boundary that surrounds all normal data, ignoring everything else.
- **How It Works:**
- **Neural Network Projector:**
  - Uses a neural network as a **transformation lens** to map input data into a new, optimized feature space.
- **The Hypersphere:**
  - During training, the model learns to pull all normal data points toward a single, pre-defined center point.
  - The objective is to **minimize the distance** of all normal points from this center.
- **The Result:**
  - A tight boundary or **"fence" (a hypersphere)**—is formed around the condensed cluster of normal data.
- **How It Detects Anomalies:**
- For any new data point:
  - It is mapped through the neural network "lens."
  - The model calculates its **Euclidean distance from the center**.
- **The Anomaly Score is this DISTANCE.**
- Points inside the fence are normal. Points far outside are anomalies.

Why These Three Models?

- I chose these three because they represent distinct and complementary approaches to anomaly detection
- VAE:
  - Modern evolution of autoencoders - uses **probability distributions** for better generalization
- GANomaly:
  - Cutting-edge approach - uses **adversarial training** to catch subtle pattern changes
- Deep SVDD
  - Completely different path - learns **direct boundaries** instead of reconstructing data

Methodology

- **Dataset: CIC-IDS2017 - The Realistic Benchmark**
- **Why I chose CIC-IDS2017 as my real-world dataset?**
  - Simulates a real corporate network over 5 days with diverse, modern attacks (DDoS, PortScan, Infiltration, etc.)
- **What does the data look like?**
  - **Format:** Tabular data (CSV) - 79 features describing network flows
  - **Each row:** A complete **bidirectional communication session** between two endpoints known as a network flow.
  - **Label:** "BENIGN" or specific attack type
  - **Benefit:** Provides a higher-level view of network activity compared to raw packet data

CIC-IDS-2017

	A	B	C	D	E	F	G	H	I	J	K	L	CA
1	Destination	Flow Duration	Total Fwd Packets	Total Backw	Total Length o	Total Length	Fwd Packet L	Fwd Packet	Fwd Packet Length I	Fwd Packe	Bwd Packe	Bwd Pack	Label
2	54865	3	2	0	12	0	6	6	6	0	0	0	BENIGN
3	55054	109	1	1	6	6	6	6	6	0	6	6	BENIGN
4	55055	52	1	1	6	6	6	6	6	0	6	6	BENIGN
89139	80	5016608	4	0	24	0	6	6	6	0	0	0	DDoS
89140	80	179305	3	5	26	11601	20	0	8.666666667	10.2632	5840	0	DDoS
89141	80	4950864	5	0	30	0	6	6	6	0	0	0	DDoS

Destination Port	Flow Duration	Total FwdPackets	Total Backward Packets
Total Length of Fwd Packets	Total Length of BwdPackets	Fwd Packet Length Max	Fwd Packet Length Min
Fwd Packet Length Mean	Fwd Packet Length Std	Bwd Packet Length Max	Bwd Packet Length Min
Bwd Packet Length Mean	Bwd Packet Length Std	Flow Bytes/s	Flow Packets/s
Flow IATMean	Flow IAT Std	Flow IAT Max	Flow IAT Min
Fwd IAT Total	Fwd IAT Mean	Fwd IAT Std	Fwd IAT Max
Fwd IAT Min	Bwd IAT Total	Bwd IAT Mean	Bwd IAT Std
Bwd IAT Max	Bwd IAT Min	Fwd PSHFlags	Bwd PSH Flags
Fwd URGFlags	Bwd URG Flags	Fwd Header Length	Bwd Header Length
Fwd Packets/s	Bwd Packets/s	Min Packet Length	Max Packet Length
Packet Length Mean	Packet Length Std	Packet Length Variance	FINFlag Count
SYNFlag Count	RSTFlag Count	PSH Flag Count	ACK Flag Count
URG Flag Count	CWEFlag Count	ECEFlag Count	Down/Up Ratio
Average Packet Size	Avg Fwd Segment Size	Avg Bwd Segment Size	Fwd Header Length
Fwd Avg Bytes/Bulk	Fwd Avg Packets/Bulk	Fwd Avg Bulk Rate	Bwd Avg Bytes/Bulk
Bwd Avg Packets/Bulk	Bwd Avg Bulk Rate	Subflow Fwd Packets	Subflow Fwd Bytes
Subflow Bwd Packets	Subflow Bwd Bytes	Init_Win_bytes_forward	Init_Win_bytes_backward
act_data_pkt_fwd	min_seg_size_forward	Active Mean	Active Std
Active Max	Active Min	Idle Mean	Idle Std
Idle Max	Idle Min		

Name of Files	Day Activity	Attacks Found
Monday-WorkingHours.pcap_ISCX.csv	Monday	Benign (Normal human activities)
Tuesday-WorkingHours.pcap_ISCX.csv	Tuesday	Benign, FTP-Patator, SSH-Patator
Wednesday-workingHours.pcap_ISCX.csv	Wednesday	Benign, DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS slowloris, Heartbleed
Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv	Thursday	Benign, Web Attack – Brute Force, Web Attack – Sql Injection, Web Attack – XSS
Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv	Thursday	Benign, Infiltration
Friday-WorkingHours-Morning.pcap_ISCX.csv	Friday	Benign, Bot
Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv	Friday	Benign, PortScan
Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv	Friday	Benign, DDoS

Methodology

- **Data Preprocessing Pipeline:**
  - **Cleaning:** Removed 308,381 duplicate records, handled missing values
  - **Feature Selection:** Dropped 8 zero-variance features
  - **Normalization:** MinMaxScaler applied **using only normal training data**
  - **Label Processing:** Converted to binary classification (Normal=0, Attack=1)
  - **Final Features:** 70 selected network flow characteristics
  - **Train-Test Split:** 70% normal traffic for training, 30% mixed traffic for testing
- **Total After Preprocessing:** 2,522,362 network flows
- **Training (Normal Only):** 1,467,538 sample
- **Test Set:** 756,709 samples
  - **Normal in Test:** 628,946 (83.1%)
  - **Anomalies in Test:** 127,763 (16.9%)
- **Anomaly Ratio:** 16.88%

Methodology

- For fair comparison setup, kept a consistent parameters across all 3 models
  - **Architecture:** Same latent dimension (64), hidden layers [256, 128]
  - **Training:** Same samples, batch size (256), epochs (50)
  - **Optimization:** Same optimizer (Adam), learning rate (0.001)
  - **Randomness:** Fixed random seed (42)

- **Why this matter?:**
  - Ensures performance differences reflect **core algorithms**, not tuning
  - Prevents any model from gaining unfair advantages
  - Provides **reproducible and consistent** benchmarks
  - Focuses on inherent strengths of each philosophical approach

Methodology

- After, I trained each the model on the training data being only normal

Methodology

- Threshold optimisation
  - Models output anomaly scores, not binary decisions
  - Hence, we need a threshold to define where to draw the line between normal and attack
  - Default thresholds are overly conservative - designed to minimize false positives, resulting in high precision but poor recall (missing many attacks).

- Why threshold optimisation is essential?
  - Precision vs. Recall Trade-off:
  - High threshold: Few false alarms, but miss many attacks
  - Low threshold: Catch most attacks, but many false alarms
  - Critical for practical cybersecurity operations where both matter
  - The models are optimized for practical detection so better balance between catching threats and managing false alarms

Methodology

- My systemic approach:
  - Tested 5 evenly distributed thresholds
  - Calculate precision, recall and F1-score for each
  - Identified the optimal threshold that **maximize F1-Score**, providing the most practical operational point

- Why precisely 5 thresholds?
  - **Computationally efficient** and practical for deployment
  - **Covers the full score spectrum** without over-engineering
  - Provides **clear precision-recall trade-off analysis**
  - More **interpretable and reproducible** than complex optimization
  - **Sufficient resolution** to find the optimal operational point

The results

- Accuracy can be misleading with imbalanced data (83% normal, 17% attacks)
- Hence we need metrics that reveal the true detection capability

- Key Metrics:
  - **Precision:** When the model flags an attack, how often is it correct? *(Low false alarms)*
  - **Recall:** What percentage of real attacks did the model detect? *(Low missed attacks)*
  - **F1-Score:** Harmonic mean of precision & recall - the **balanced metric**
  - **ROC-AUC:** Model's overall ability to distinguish normal vs attack

- The default threshold before optimisation gave high accuracy (0.85 – 0.90) and high precision but poor recall, hence missed attacks

The results

- After threshold optimization : Performance comparison on CIC-IDS-2017

Model	ROC-AUC	F1-Score	Precision	Recall	Accuracy
VAE	0.853	**0.629**	0.655	0.605	**0.880**
GANomaly	0.790	0.466	0.389	0.582	0.778
Deep SVDD	**0.862**	0.591	0.639	0.549	0.867

The result

Model	ROC-AUC	F1-Score	Precision	Recall	Accuracy
VAE	0.853	**0.629**	0.655	0.605	**0.880**
GANomaly	0.790	0.466	0.389	0.582	0.778
Deep SVDD	**0.862**	0.591	0.639	0.549	0.867

- **1. VAE: The Most Balanced Performer**
  - **Highest F1-Score (0.629):** It achieves the best overall balance between precision and recall.
  - **Strong Recall (0.605):** It successfully detects over 60% of attacks.
  - **Conclusion:** The **most reliable and practical choice** for general use.
- **2. Deep SVDD: The Best Discriminator**
  - **Highest ROC-AUC (0.862):** It is the best model at fundamentally separating normal traffic from attacks.
  - **Good Precision (0.639):** When it flags an anomaly, it is often correct.
  - **Conclusion:** Excellent for environments where **computational efficiency and clear separat**ion are key.
- **3. GANomaly: The High-Sensitivity Model**
  - **Significant Recall (0.582):** It can detect a wide range of attacks.
  - **Major Trade-Off:** This comes at the cost of **low precision (0.389)**, leading to many false alarms.
  - **Conclusion:** Useful in **high-security settings** where missing an attack is worse than investigating false alerts.

The results

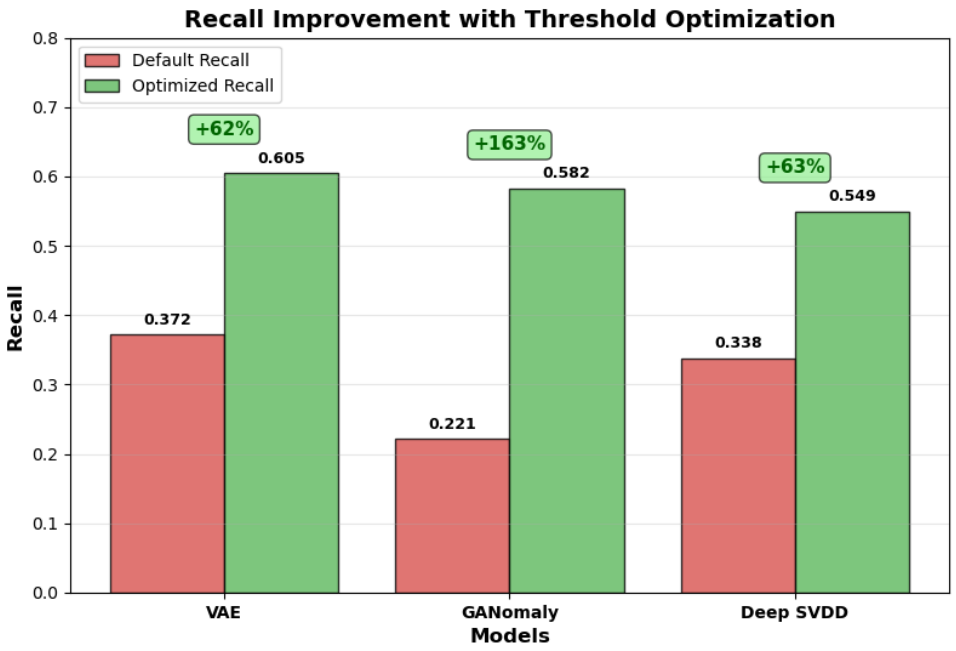
- Threshold optimisation effects:

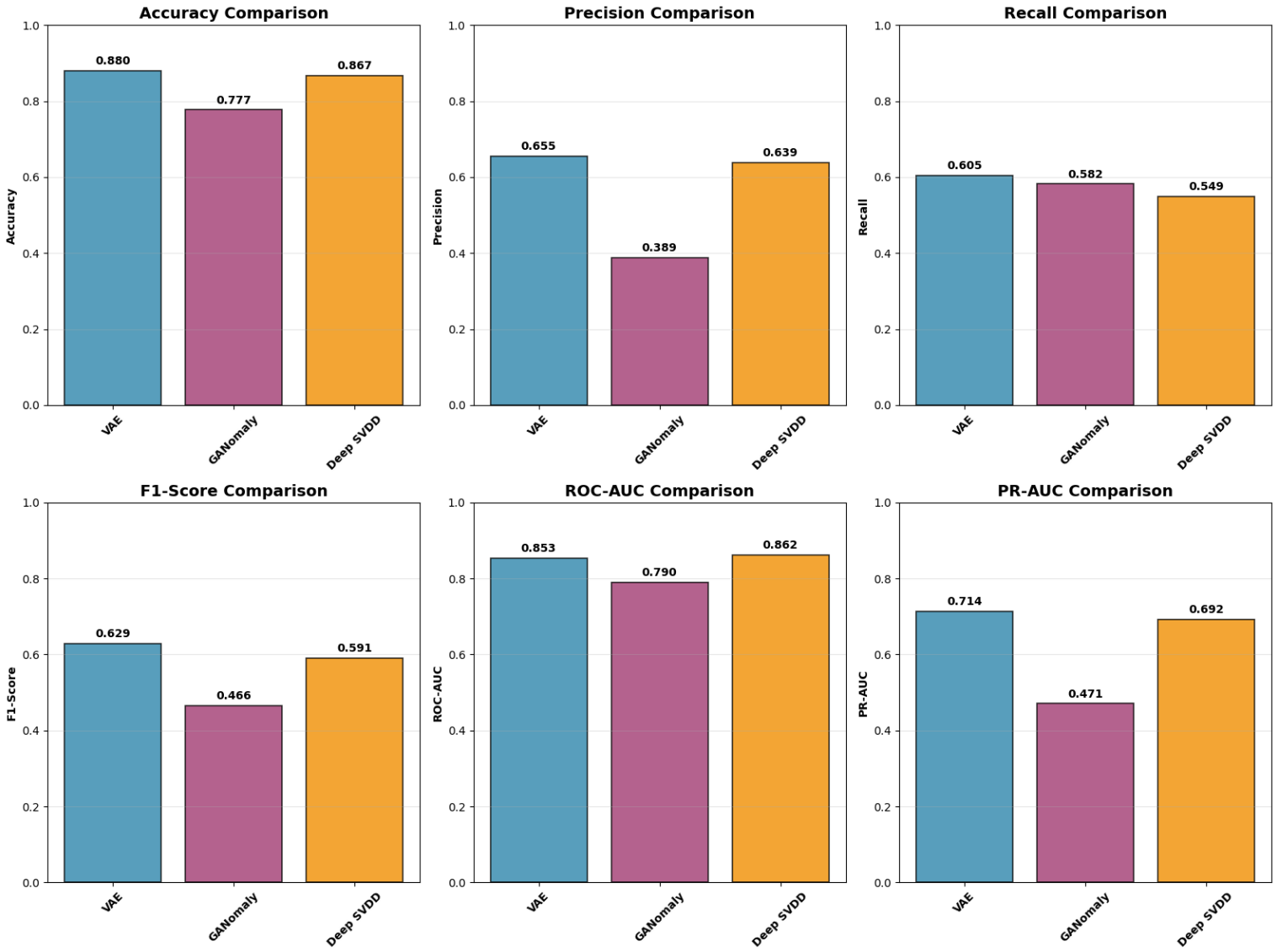
Model	Default Recall	Optimized Recall	Improvement
VAE	0.372	0.605	**+63%**
GANomaly	0.221	0.582	**+163%**
Deep SVDD	0.338	0.549	**+62%**

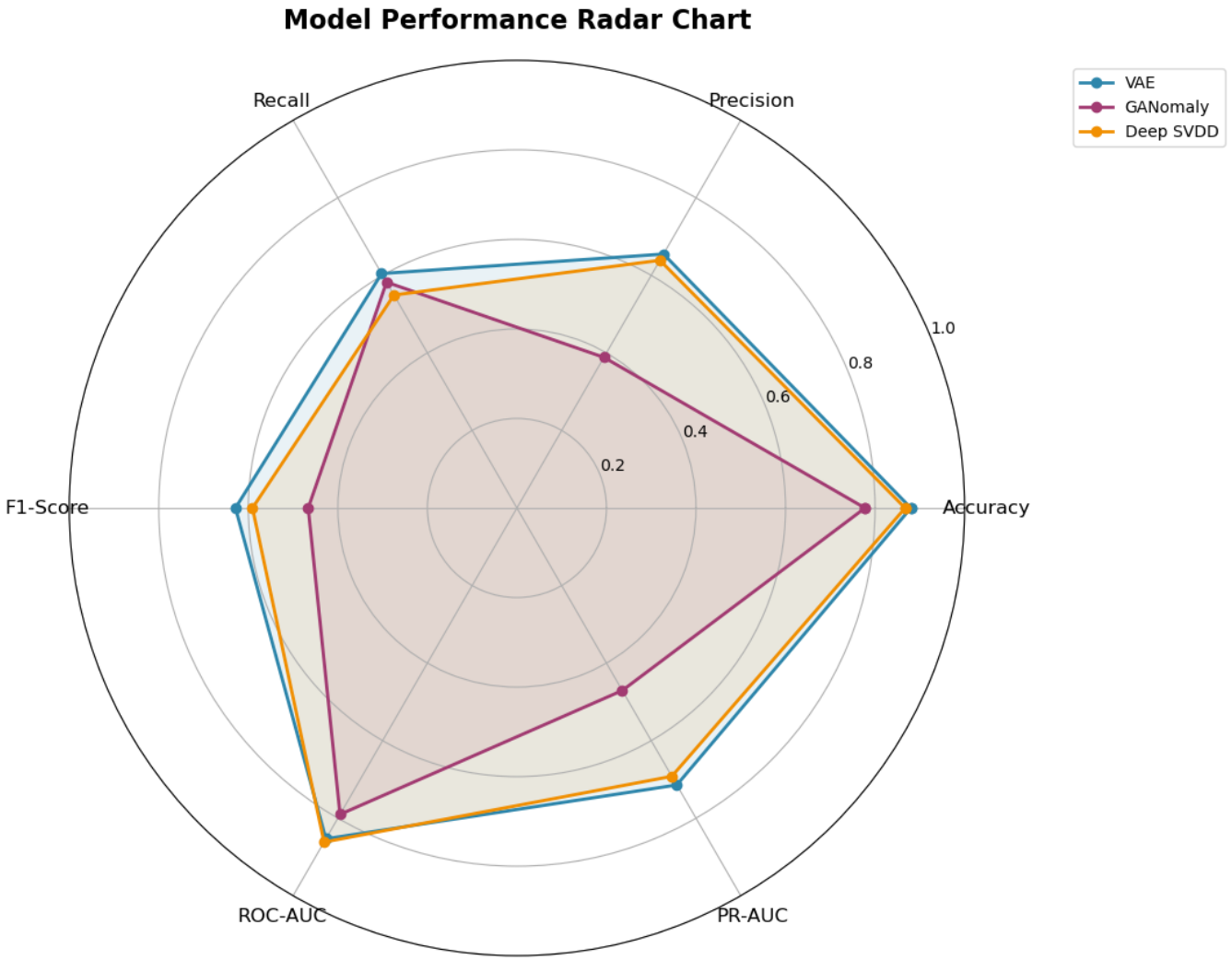
The results

Model	Default Recall	Optimized Recall	Improvement
VAE	0.372	0.605	**+63%**
GANomaly	0.221	0.582	**+163%**
Deep SVDD	0.338	0.549	**+62%**

- All models improved by >60% in recall after optimization
- **GANomaly: The High-Risk, High-Reward Model**
  - **+163% recall improvement** - the most dramatic gain
  - But: **Major precision trade-off** - many more false alarms
  - **Lesson:** Extremely sensitive to threshold settings
- **VAE: The Balanced Champion**
  - **+63% recall improvement** while maintaining good precision
  - **Most stable and reliable** performance
  - **Lesson:** Best choice for operational environments
- **Deep SVDD: The Consistent Performer**
  - **+62% improvement** with stable performance
  - **Predictable and manageable** trade-offs
  - **Lesson:** Reliable for resource-constrained deployments




Accuracy Comparison
Accuracy
0.880
0.777
0.867
Precision Comparison
Precision
0.655
0.389
0.639
Recall Comparison
Recall
0.605
0.582
0.549
F1-Score Comparison
F1-Score
0.629
0.466
0.591
ROC-AUC Comparison
ROC-AUC
0.853
0.790
0.862
PR-AUC Comparison
PR-AUC
0.714
0.471
0.692
VAE
GANomaly
Deep SVDD


Model Performance Radar Chart
Recall
Precision
F1-Score
Accuracy
ROC-AUC
PR-AUC
0.2
0.4
0.6
0.8
1.0
VAE
GANomaly
Deep SVDD

Why we saw these results?

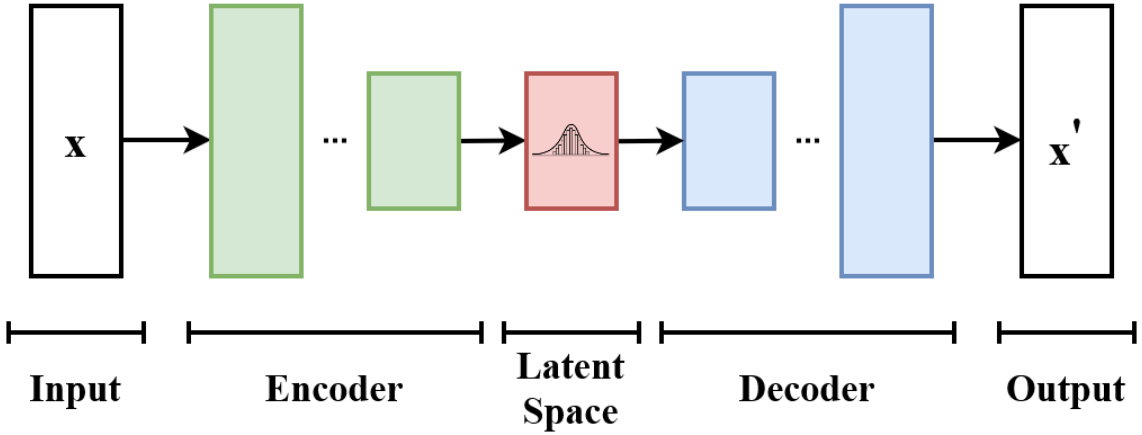
- **GANomaly's Volatility: The Price of Adversarial Training**
  - **Inherent Instability:** GAN training is famously unstable - the generator and discriminator are in constant competition
  - **High Sensitivity:** This makes the model very sensitive to hyperparameters and thresholds
  - **Result:** Dramatic performance swings (+163% recall) but major precision trade-offs
- **VAE's Stability: The Benefit of Probabilistic Design**
  - **Solid Foundation:** Probabilistic approach creates a robust, smooth latent space
  - **Better Generalization:** Learns the "essence" of normal data rather than memorizing it
  - **Result:** Consistent, balanced improvements across all metrics
- **Deep SVDD's Efficiency: The Power of Focused Learning**
  - **Single Objective:** Directly minimizes the hypersphere volume - no reconstruction complexity
  - **Computational Stability:** More predictable optimization landscape
  - **Result:** Stable, reliable improvements without dramatic swings
- **The Pattern:** More complex training (adversarial) → Higher potential but less stability

Recommendations

- **Actionable Recommendations for Security Teams:**
  - **For Most Companies:** Use **VAE**. Its stability and balanced performance prevent alert fatigue and provide reliable detection.
  - **For High-Security, Resource-Rich Environments:** Consider **GANomaly**. If you can afford to investigate many false positives, its high recall can help catch subtle, advanced threats.
  - **For Large-Scale Monitoring with Limited Resources:** Use **Deep SVDD**. Its computational efficiency and fast inference make it ideal for scanning vast amounts of traffic.

Recommendation

- Based on our results, I recommend the Variational Autoencoder (VAE) for most practical deployments, as it provides the most reliable balance between detecting attacks and maintaining a manageable false alarm rate.



Discussion & future

- **Key Insights:**
  - **No Single "Best" Model:** The optimal choice depends on the specific network environment, data type, and security priorities. Our results are a **practical guide**, not a definitive answer for all scenarios.
  - **The Power of Ensembles:** The most robust future system might **combine these models**. For example, using Deep SVDD as a fast first-pass filter and VAE for deeper analysis could yield superior performance.
- **Future Work:**
  - **Expand Evaluation:** Test on more diverse datasets from different organizations to ensure broader applicability.
  - **Enhance Explainability:** Investigate explainable AI (XAI) to tell security analysts *why* a flow was flagged, which is critical for trust and effective response.
  - **Towards Continuous Defense:** The ultimate goal is a system that runs **autonomously in real-time**, learning normal behavior and flagging anomalies as they occur on the network.

Limitations

- **1. Dataset Scope:**
  - **Single Benchmark:** Our evaluation was conducted on the CIC-IDS2017 dataset. While it's a realistic benchmark, performance may vary across different network environments (e.g., a university network vs. a corporate data center).
- **2. Operational Overhead:**
  - **Computational Cost:** This study focused on detection accuracy. A full deployment analysis would require measuring the computational cost, latency, and scalability of each model in a real-time streaming environment.
  - **Concept Drift:** Networks evolve over time. Our static evaluation does not address the challenge of continuous retraining to adapt to new "normal" behavior.
- **3. Architectural Scope:**
  - **Focused Comparison:** We compared three specific architectures. Other promising deep learning models (e.g., Transformers for sequences, Graph Neural Networks for network connections) were outside the scope of this study including ensemble methods.

Ethics and Social implications

- **1. Privacy and Anonymization:**
  - We used exclusively **public, pre-anonymized datasets**. No private user or organizational data was accessed, ensuring compliance with privacy standards.
- **2. Potential for Misuse:**
  - While our research is for **defensive cybersecurity**, the same technology could potentially be misused for mass surveillance or invasive monitoring. We strongly advocate for its use only within ethical and legal frameworks.
- **3. Algorithmic Bias and Fairness:**
  - Models learn from historical data. If this data reflects existing biases (e.g., flagging traffic from certain countries or departments as anomalous), the system could **amplify these biases**. Continuous monitoring for discriminatory outcomes is essential.
- **4. Transparency and Accountability:**
  - Deep learning models can be "black boxes." The significant impact of **threshold optimization** underscores the need for transparent operational parameters. Security analysts need to understand *why* an alert was generated to make informed decisions.

Tools

- I implemented the models using Python with TensorFlow/Keras for deep learning, alongside NumPy, pandas, scikit-learn, matplotlib, seaborn, and standard OS libraries for data processing, evaluation, and visualization.
- The dataset used was CIC-IDS-2017 obtained from Kaggle: https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset

Code availability

- **Code Repository:**
  GitHub: https://github.com/adwanis/AIML339-anomaly-detection

Thank you